OCR Project Documentation

1. Project Overview

The goal of this project is to automate the extraction of handwritten data from a scanned/photographed 3-page form into a structured CSV file. The form is primarily in the Hindi language, and respondents fill it out using a pen.

The tool processes the filled forms, aligns them perfectly against a blank template, extracts specific fields (text, checkboxes, and circled numbers), and appends the extracted data to an output file (`output.csv`).

2. Technology Stack

- **Python 3**: Core language used for the logic.
- **PyMuPDF (fitz)**: Used for rendering PDF files into high-resolution images for processing.
- **OpenCV (`cv2`)**: Used for image alignment, cropping, and contour detection (checkboxes and circles).
- **Google GenAI SDK (`google-genai`) & Gemini API**: Uses the `gemini-3.5-flash` multimodal model as the primary engine for high-accuracy handwriting recognition in text fields (headers and free-text).
- **EasyOCR**: A deep-learning-based OCR engine used as a fallback for extracting handwritten text if the Gemini API is unavailable or fails. Supports mixed Hindi/English.
- **Pandas & NumPy**: Used for matrix operations, structuring the final extracted row, and saving to CSV.

3. Project File Structure

The project consists of the following key Python files:

`main.py`

This is the entry point of the application. It orchestrates the entire pipeline:

- Parses command-line arguments (target PDF files).
- Converts the blank template PDF to images.
- Loops through each page of the filled PDF, calling `align_page()` to align it.
- Calls specific reading functions (like `read_text`, `read_checkbox`, `read_circled_number`) from `detect.py` using bounding boxes defined in `config.py`.
- Assembles the final dictionary of results and appends it to `output.csv`.

`config.py`

This file acts as a central registry for all coordinates (bounding boxes). Since the PDFs are rendered at a 3x zoom (resulting in 1786 x 2526 pixel images), all coordinates here are absolute pixel coordinates corresponding to that resolution.

- Defines bounding boxes for header fields (`Date`, `Khoji`, `City`, `Name`) on all 3 pages.
- Defines bounding boxes for "Yes/No" checkboxes (Page 1, Q1 and Q2).
- Defines bounding boxes for the rows containing circled numbers 1-10 (Questions 3-13).

`align.py`

Handles the translation of PDFs to images and corrects scanning imperfections:

- `pdf_to_images()`: Uses PyMuPDF with a custom scale matrix to render the PDF pages into BGR numpy arrays.
- `align_page()`: Corrects rotation, skew, and zoom of the filled form relative to the blank template. It does this by extracting ORB features from both images, finding matches, and computing a Homography matrix to warp the filled image perfectly over the template.

`detect.py`

Contains the computer-vision and OCR extraction logic for individual crops:

- `read_text()` & `read_text_header()`: First attempts to send the cropped image to the Gemini API (`gemini-3.5-flash`) via the `google-genai` SDK for highly accurate transcription. If the API key (`GEMINI_API_KEY`) is missing or the request fails, it falls back to local EasyOCR with specific preprocessing variants.
- `read_checkbox()`: Converts the crop to grayscale, applies a binary threshold, and checks the percentage of dark pixels. If the box is filled heavily compared to a blank threshold, it counts as " checked".
- `read_circled_number()`: Takes a row crop containing numbers 1-10. It compares the filled crop against the blank template crop to highlight the pen marks using Absolute Difference. It then finds contours, filters them by area and aspect ratio (to find the circle), and calculates which number was circled based on the x-coordinate of the circle's center.

4. Extraction Process Flow

1. **Input Registration**: The user runs `python main.py <Filled_Form.pdf>`.
2. **PDF Rendering**: The blank template (`Nisarg form_Nov 2025.pdf`) and the filled PDF are rendered into high-resolution images.
3. **Image Alignment**: For each of the 3 pages, OpenCV matches structural features of the filled page to the blank page, mathematically warping the filled page so it exactly overlays the blank one. This ensures that coordinates in `config.py` are perfectly accurate regardless of how the form was scanned.
4. **Field Extraction (Page 1)**:
    - The top headers (Date, Khoji No, City, Name) are cropped and sent to EasyOCR.
    - Q1 and Q2 are cropped specifically at the "Han" (Yes) and "Na" (No) boxes. Pixel density determines the answer.
    - Q3-Q12 rows are cropped. Contour detection finds the hand-drawn circle and maps it to a score (1-10).
5. **Field Extraction (Page 2)**:
    - Headers are scanned (only updated if Page 1's headers were empty).
    - Q1-Q13 rows are processed for circled numbers.
    - Q14's large text box is cropped and sent to EasyOCR.
6. **Field Extraction (Page 3)**:
    - Q1-Q13 rows are processed for circled numbers.
7. **Data Export**: A Pandas DataFrame is constructed with 43 columns (including `Source_File`) and appended to `output.csv`.

5. Known Limitations & Future Improvements

- **Handwriting Recognition & API Dependency**: The pipeline now uses the Gemini API (

`gemini-3.5-flash`) for highly accurate handwriting recognition, resolving previous issues with EasyOCR on cursive text. However, this introduces a dependency on an internet connection and a valid `GEMINI_API_KEY`. Without the key, the system falls back to EasyOCR, which may struggle with cursive or mixed Hindi/English handwriting.

- **Improvement Path**: Future improvements could include fine-tuning local, offline LLMs for transcription to remove the cloud dependency while maintaining the accuracy of the Gemini API.